

Contents

1	System Overview	2
1.1	Key Features	2
2	Installation and Setup	2
2.1	System Requirements	2
2.2	Required Dependencies	2
2.3	File Structure Setup	3
3	Basic Operation Guide	3
3.1	Step 1: Launch the Application	3
3.2	Step 2: Model Configuration	3
3.2.1	Loading the Model File	3
3.2.2	Adjusting Detection Sensitivity	3
3.3	Step 3: Live Webcam Detection	4
3.3.1	Webcam Controls	4
4	CCTV Camera Integration	4
4.1	IP Camera Configuration	4
4.2	Camera Configuration File	5
5	Model File Management	5
5.1	Handling Variable Model Paths	5
6	Advanced Configuration	6
6.1	Detection Parameters	6
6.2	Network Configuration for CCTV	7
6.2.1	Common CCTV Camera URLs	7
6.2.2	Firewall Configuration	7
7	Troubleshooting Guide	7
7.1	Common Issues and Solutions	7
8	Deployment Checklist	7
8.1	Pre-deployment	7
8.2	Post-deployment	8
9	Conclusion	8

1 System Overview

The Safety Helmet and PPE Vest Detection System is an AI-powered computer vision application that monitors workplace safety by detecting whether workers are wearing proper safety equipment. The system uses YOLOv8 object detection to identify persons, safety helmets, and PPE vests in real-time, triggering audio alarms when safety violations are detected.

1.1 Key Features

- Real-time detection of safety helmets and PPE vests
- Audio alarm system for safety violations
- Support for webcam and CCTV camera integration
- Image and video processing capabilities
- Configurable detection sensitivity
- Multi-camera monitoring support

2 Installation and Setup

2.1 System Requirements

Component	Requirement
Operating System	Windows 10/11 (for audio alarm functionality)
Python Version	3.8 or higher
RAM	Minimum 8GB (16GB recommended)
Storage	At least 2GB free space
Camera	USB webcam or IP camera access

Table 1: System Requirements

2.2 Required Dependencies

Install the following Python packages using pip:

```
1 pip install tkinter opencv-python pillow ultralytics numpy threading  
   queue winsound
```

Listing 1: Basic Dependencies Installation

For CCTV integration, also install:

```
1 pip install requests urllib3 rtsp-client
```

Listing 2: CCTV Integration Dependencies

2.3 File Structure Setup

Organize your project files as follows:

```

1 safety_detection_system/
2     safety_helmet_detection.py      # Main application file
3     models/                        # Model directory
4         best.pt                    # YOLOv8 trained model
5         backup_models/             # Backup model files
6     config/                        # Configuration files
7         camera_config.json         # CCTV camera settings
8         detection_config.json      # Detection parameters
9     logs/                          # System logs
10    recordings/                     # Saved frames and videos
11    documentation/                 # User manuals and guides

```

Listing 3: Project Directory Structure

3 Basic Operation Guide

3.1 Step 1: Launch the Application

Run the main application:

```
1 python safety_helmet_detection.py
```

Listing 4: Application Launch

Upon startup, you'll see a mode selection dialog with four options:

- **Live Webcam Detection:** Real-time monitoring using connected webcam
- **Detect in Images:** Process individual image files
- **Detect in Videos:** Process video files
- **All Detection Modes:** Access to all functionality

3.2 Step 2: Model Configuration

3.2.1 Loading the Model File

1. The system looks for `best.pt` at the default path: `C:\Users\princ\OneDrive\Desktop\best.pt`
2. If the model file is in a different location, click "**Browse**" to select the correct path
3. Click "**Reload Model**" to load the selected model
4. Verify the status shows "**Model loaded successfully!**"

3.2.2 Adjusting Detection Sensitivity

- Use the **Confidence Threshold** slider to adjust detection sensitivity
- Lower values (0.1-0.3): More detections, potentially more false positives
- Higher values (0.7-0.9): Fewer detections, higher accuracy
- Recommended setting: **0.5** for balanced performance

3.3 Step 3: Live Webcam Detection

1. Select "Live Webcam Detection" or click "Start Webcam" in all modes
2. A separate webcam window will open showing the live feed
3. The system will automatically detect and highlight:
 - **Green boxes:** Safety helmets detected
 - **Red boxes:** Persons without helmets
 - **Blue boxes:** PPE vests detected

3.3.1 Webcam Controls

- Press 'S' to save the current frame
- Press 'Q' to quit webcam mode
- **Audio Alarm:** Automatically triggers when a person is detected without a helmet

4 CCTV Camera Integration

4.1 IP Camera Configuration

To integrate with CCTV cameras, modify the webcam initialization code to support IP cameras:

```
1 def connect_ip_camera(self, ip_address, username=None, password=None,
2   port=554):
3     """Connect to IP camera using RTSP protocol"""
4     try:
5         if username and password:
6             rtsp_url = f"rtsp://{username}:{password}@{ip_address}:{
7 port}/stream"
8         else:
9             rtsp_url = f"rtsp://{ip_address}:{port}/stream"
10
11         # Alternative HTTP stream for some cameras
12         http_url = f"http://{ip_address}/video.cgi"
13
14         self.cap = cv2.VideoCapture(rtsp_url)
15
16         if not self.cap.isOpened():
17             # Try HTTP stream as fallback
18             self.cap = cv2.VideoCapture(http_url)
19
20         if self.cap.isOpened():
21             self.log_result(f"Connected to IP camera: {ip_address}")
22             return True
23         else:
24             self.log_result(f"Failed to connect to IP camera: {
25 ip_address}")
26             return False
27
28     except Exception as e:
```

```

26     self.log_result(f"IP camera connection error: {str(e)}")
27     return False

```

Listing 5: IP Camera Connection Function

4.2 Camera Configuration File

Create config/camera_config.json for managing multiple CCTV cameras:

```

1  {
2      "cameras": [
3          {
4              "id": "camera_01",
5              "name": "Warehouse Entrance",
6              "ip": "192.168.1.100",
7              "port": 554,
8              "username": "admin",
9              "password": "admin123",
10             "stream_path": "/stream1",
11             "protocol": "rtsp",
12             "active": true
13         },
14         {
15             "id": "camera_02",
16             "name": "Loading Dock",
17             "ip": "192.168.1.101",
18             "port": 8080,
19             "username": "security",
20             "password": "secure456",
21             "stream_path": "/video.cgi",
22             "protocol": "http",
23             "active": true
24         }
25     ],
26     "detection_settings": {
27         "confidence_threshold": 0.5,
28         "alarm_cooldown": 2,
29         "save_violations": true,
30         "log_all_detections": false
31     }
32 }

```

Listing 6: Camera Configuration JSON

5 Model File Management

5.1 Handling Variable Model Paths

Since the `best.pt` file location varies between systems, implement flexible path management:

```

1  class ModelPathManager:
2      def __init__(self):
3          self.possible_paths = [
4              "models/best.pt",          # Relative path in
project

```

```

5         "./best.pt",                                # Current directory
6         "C:/models/best.pt",                        # System models folder
7         os.path.expanduser("~/models/best.pt"),     # User home
directory
8         "D:/AI_Models/best.pt"                      # Alternative drive
9     ]
10
11     def find_model_file(self):
12         """Search for model file in common locations"""
13         for path in self.possible_paths:
14             if os.path.exists(path):
15                 return path
16         return None
17
18     def auto_load_model(self):
19         """Automatically load model from available locations"""
20         model_path = self.find_model_file()
21         if model_path:
22             self.model_path_var.set(model_path)
23             self.load_model()
24             return True
25         else:
26             messagebox.showwarning(
27                 "Model Not Found",
28                 "Please browse and select your best.pt model file"
29             )
30         return False

```

Listing 7: Model Path Manager Class

6 Advanced Configuration

6.1 Detection Parameters

Create config/detection_config.json for fine-tuning:

```

1 {
2     "detection_parameters": {
3         "confidence_threshold": 0.5,
4         "iou_threshold": 0.45,
5         "max_detections": 100,
6         "image_size": 640
7     },
8     "alarm_settings": {
9         "enable_audio": true,
10        "alarm_frequency": 1000,
11        "alarm_duration": 500,
12        "cooldown_period": 2
13    },
14    "logging": {
15        "save_violations": true,
16        "log_level": "INFO",
17        "max_log_size": "10MB",
18        "retention_days": 30
19    },
20    "performance": {
21        "frame_skip": 1,

```

```

22     "resize_factor": 1.0,
23     "gpu_acceleration": true
24 }
25 }

```

Listing 8: Detection Configuration

6.2 Network Configuration for CCTV

6.2.1 Common CCTV Camera URLs

- Hikvision: `rtsp://username:password@ip:554/Streaming/Channels/101`
- Dahua: `rtsp://username:password@ip:554/cam/realmonitor?channel=1&subtype=0`
- Axis: `rtsp://username:password@ip:554/axis-media/media.amp`
- Generic RTSP: `rtsp://username:password@ip:554/stream`

6.2.2 Firewall Configuration

- Open ports 554 (RTSP), 80 (HTTP), 8080 (Alternative HTTP)
- Ensure cameras are on the same network or accessible via VPN
- Configure port forwarding if accessing cameras remotely

7 Troubleshooting Guide

7.1 Common Issues and Solutions

Issue	Error Message	Solution
Model Loading	"Model file not found"	Use Browse button to locate best.pt file. Check for special characters in path.
Camera Connection	"Could not open web-cam"	Check if another app is using camera. Try different camera indices (0,1,2).
CCTV Integration	"Failed to connect to IP camera"	Verify IP, username, password. Test with VLC first. Check network/firewall.
Performance	Slow detection/High CPU	Reduce confidence threshold. Lower resolution. Enable GPU acceleration.

8 Deployment Checklist

8.1 Pre-deployment

Test model accuracy with sample images from your environment

Verify all CCTV cameras are accessible and configured

Set appropriate confidence thresholds for your use case

Test alarm functionality and volume levels

Configure automatic startup if required

8.2 Post-deployment

Monitor system performance for the first week

Collect feedback from safety personnel

Adjust detection parameters based on real-world performance

Set up regular model updates and maintenance schedule

Train staff on system operation and troubleshooting

9 Conclusion

This user manual provides comprehensive guidance for implementing and operating the Safety Helmet and PPE Vest Detection System in various environments, from simple webcam setups to complex multi-camera CCTV installations. The system offers flexible configuration options and robust performance for workplace safety monitoring applications.